STATE OF AI ENGINEERING

DATADOG

Over the past year, generative AI technology has matured in leaps and bounds. Teams are no longer simply wiring a single model call into a service and calling it a product. Moving from experimentation to production, they are managing model fleets, orchestration frameworks, tool calls, long prompts, retries, and multiple service boundaries. This work should look familiar: routing, life cycle management, capacity planning, cost control, and debugging across distributed systems.

However, the difference LLMs present is that model, prompt, or retrieval changes can move latency, spend, and failure rates without an obvious code change. As teams ship agents to production, the gap between a good demo and a dependable system is closed by effective evaluation and operational discipline.

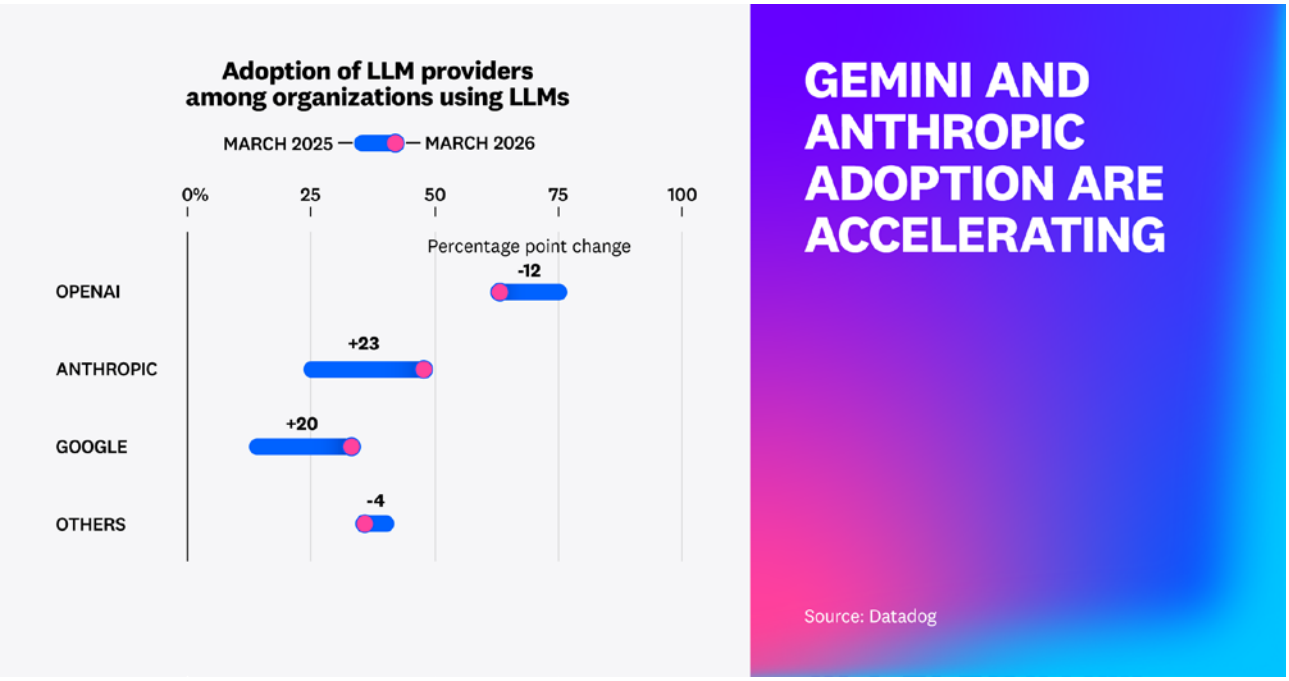
This report explores the state of AI engineering in production by looking at LLM telemetry from more than a thousand Datadog customers. Our findings suggest that while teams have meaningful opportunities to improve efficiency and reliability across model fleet management, agent design, context engineering, and cost optimization, identifying and attaining those gains can be challenging in a fast-moving ecosystem. We explore how adoption of providers, languages, and orchestration frameworks is evolving; how production workloads use context, tools, and multi-step workflows; and the common failure modes that emerge at scale.

We use the terms “AI applications” to describe production services that make LLM calls and “agents” to describe the subset that use multi-step control flow, tool execution, or multiple service calls. Some findings below apply to all AI applications in the dataset; others focus specifically on agentic workloads. Together, they show how AI engineering is taking shape in production.

FACT 1

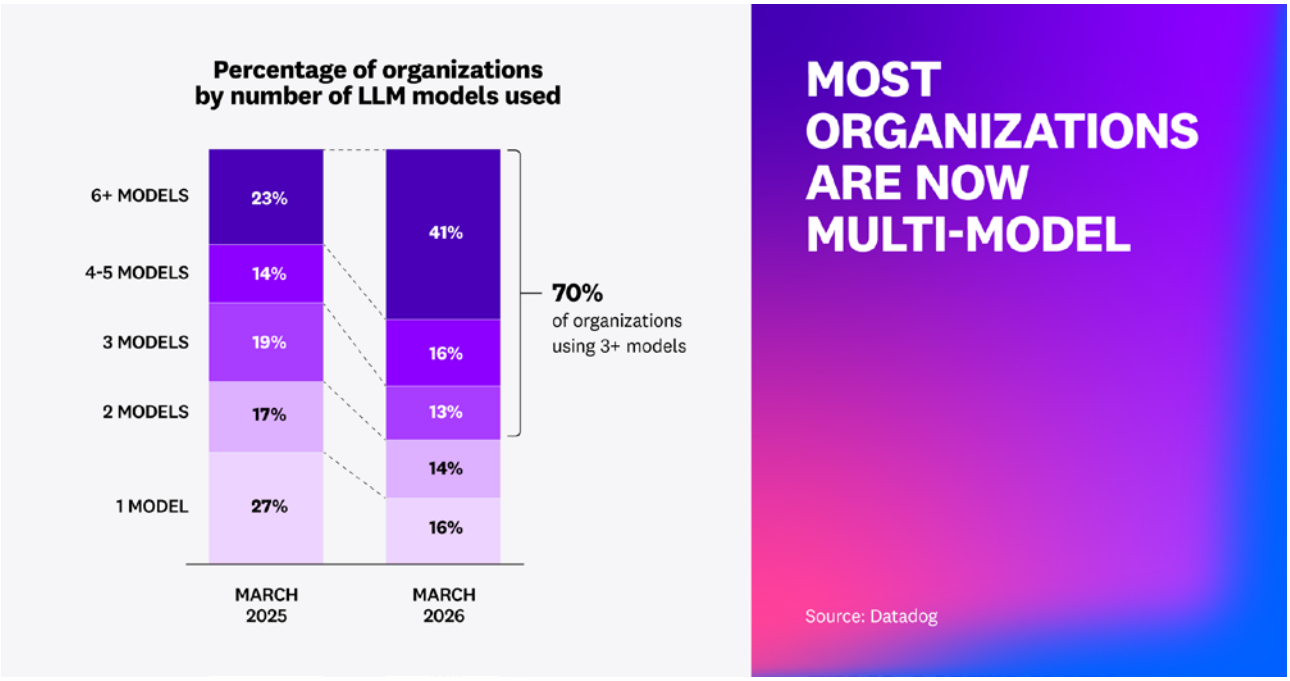
Organizations are increasingly relying on multiple models

By analyzing our dataset of customer LLM agent telemetry, we found that organizations are increasingly multi-provider, with OpenAI holding a 63% share but Google Gemini and Anthropic Claude shares gaining 20 and 23 percentage points, respectively, over the last year.



Importantly, while OpenAI's share is down from 75% a year ago, this does not mean they experienced a decline in absolute use. We found that the number of Datadog customers using OpenAI more than doubled, even as other providers grew faster.

Model diversification is showing up inside organizations as well. More than 70% of organizations now use three or more models, and the share using more than six models nearly doubled. Rather than picking a single default, teams are building model portfolios in order to use the optimal model for each workload's latency, cost, operational risk, and task requirements.



However, this multi-provider platform pattern also introduces numerous platform engineering, DevX, and compliance challenges. Juggling scattered API calls over disparate model providers and services can make it hard to iterate quickly, enforce safety and compliance standards consistently, as well as fail over gracefully when model providers throttle requests or degrade in performance and efficiency. This is why teams increasingly need to use a modular routing mechanism (such as a gateway service or a managed gateway like OpenRouter) to manage LLM requests rather than rely on direct model provider API calls throughout their environments.

The teams pulling ahead today treat inference like a pipeline, routinely evaluating, benchmarking and swapping in the right model for each stage (e.g., lightweight models for extraction and tagging, frontier models for synthesis) as costs fall and model performance evolves. By running a model gateway and maintaining an operationalized evaluation framework, teams can pick the right model for each of their use cases according to output quality, cost, and latency. Particularly, using online evaluations is critical for understanding model and agent output quality, safety, and performance in production in order to make informed model selection decisions.

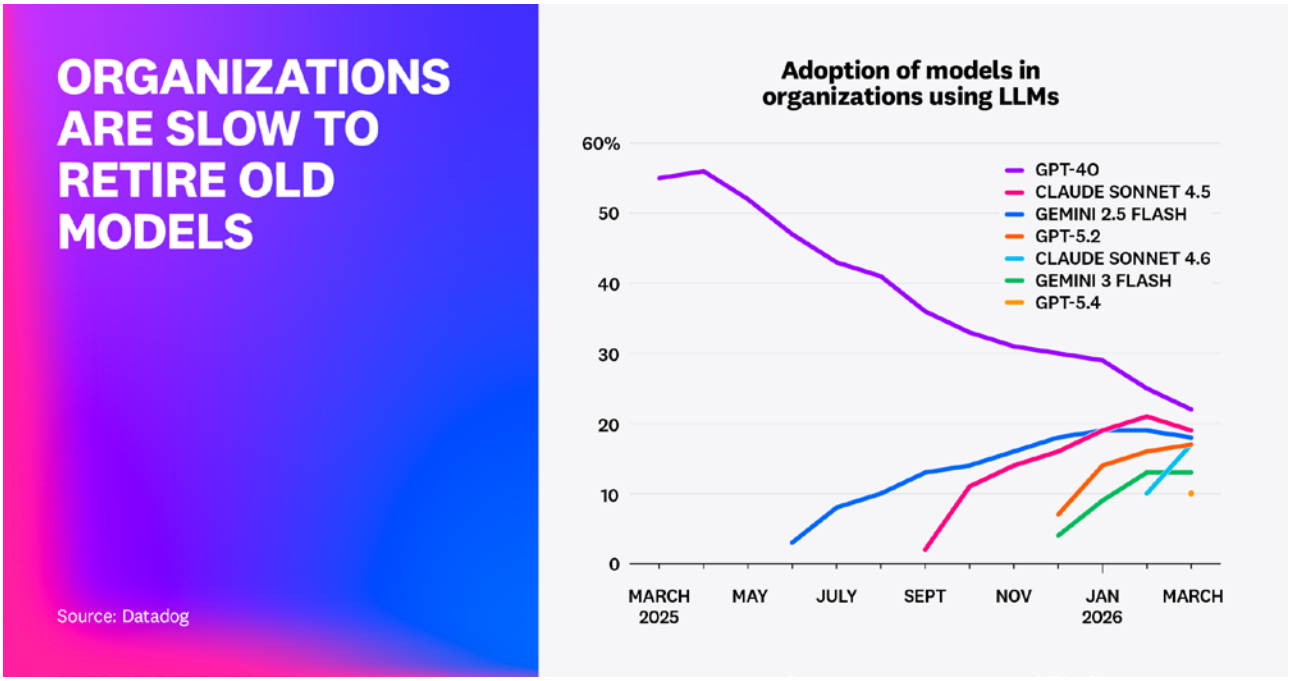
"Most teams are using multiple models in production now. Around 70% are running three or more, and that number keeps growing, with agents accelerating the trend. That's why we provide one integration to safely tap into hundreds of models, for startups and global enterprises alike. Users want to be able to switch quickly, test freely, and discover the best model for their workflows."

Alex Atallah
Co-Founder & CTO at OpenRouter

FACT 2

LLM tech debt compounds as teams quickly adopt new releases while keeping old defaults

As organizations move into multi-model environments, they also inherit the complexity of maintaining them. Our analysis of Datadog customers' model usage suggests that teams are quick to test new releases in order to stay competitive but slower to retire older models already running in production. As a result, many organizations could be adding new models faster than they are simplifying their fleets. When running agents in production, each overlapping model increases operational overhead and introduces an evaluation burden. That's why teams need to continuously validate performance and manage regressions for all the models they've invested in.



The same prompts, tools, and agent workflows can produce different results across models, which means that each additional model may introduce its own quality, latency, and cost profile. In practice, model churn becomes a governance problem.

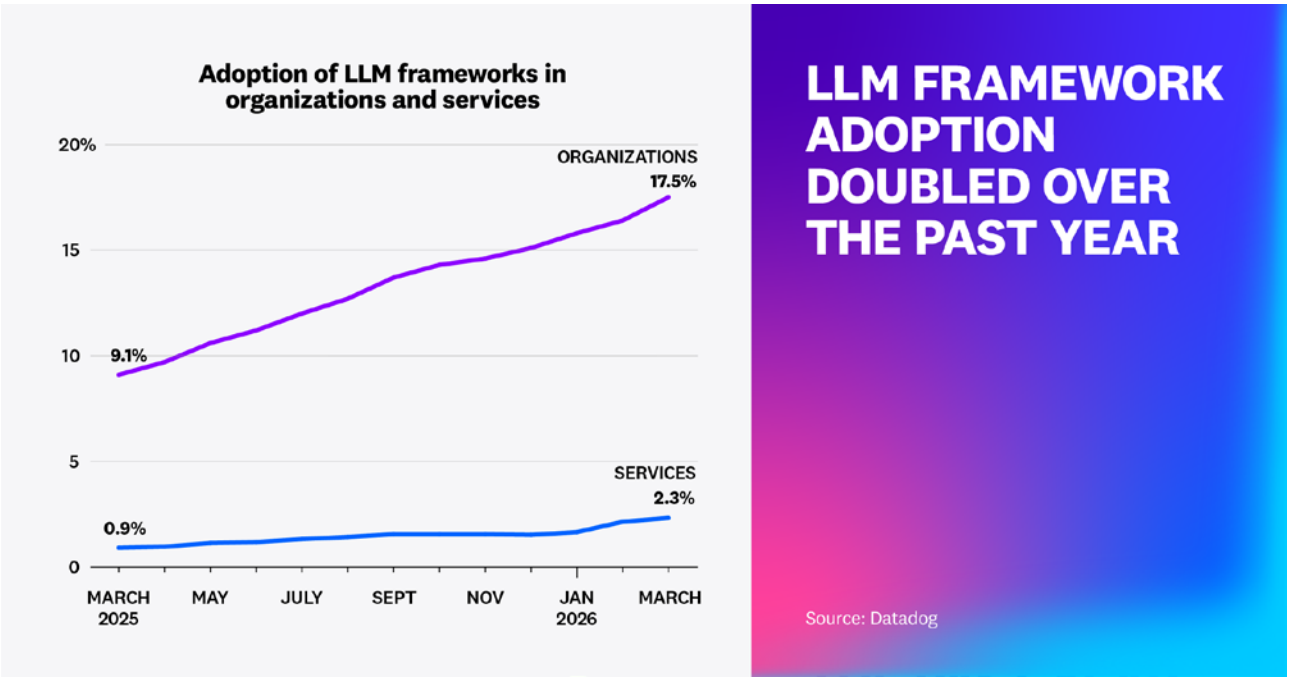
We looked at the adoption rates of seven popular models to understand how organizations are responding to new model releases. We found that teams are relatively quick to add newer models upon release; for example, Claude Sonnet 4.6 grew to 17% adoption in its first month. Still, adoption of older models such as Sonnet 4.5 and GPT-4o have declined but remained at 19% and 22%, respectively, as of March 2026—similar levels to Sonnet 4.6 and GPT-5.4. In 2026, there appears to be no clear winner for model choice, and teams are increasingly keeping multiple models in flight.

While managing multi-model systems can be handled effectively with continuous evaluation and governance, as well as efficient routing via gateways, teams will nevertheless need to manage the ongoing deprecation of older models as providers continue to sunset them. For instance, although GPT-4o was still the most common model used in the request traces our research examined for March 2026, OpenAI has already retired this model in the ChatGPT UI, making the future of its API support somewhat uncertain.

FACT 3

As agent framework adoption doubles, the need for deep telemetry also rises

Agent frameworks, such as LangChain, Pydantic AI, LangGraph, and Vercel AI SDK, accelerate development by making common patterns easy to add. Our research found that framework adoption has nearly doubled year over year in 2026, rising from more than 9% of organizations in early 2025 to almost 18% by the beginning of 2026. Similarly, the number of services using agentic frameworks more than doubled in the same time period. While frameworks speed up building, they can also introduce costly operational complexity, which requires comprehensive agent telemetry so that teams can see how agents execute and identify inefficient imported logic that can be replaced with bespoke workflows.



Notably, this framework growth pattern remained consistent across start-ups, mid-market, and enterprise-level organizations.

When teams use framework boilerplate for key patterns, agent sprawl can set in as the framework adds more steps and paths under the hood and it becomes harder for engineers to understand what's happening in the runtime. In framework-assisted AI application development, tool fan-out, retries, and branching are one import away. This can cause cost and latency to drift upward and make failures more difficult to reproduce. That's why it's critical for teams to collect comprehensive agent telemetry to understand how agents execute in practice, diagnose unexpected behaviors, and identify where workflows diverge from intended outcomes—signals to build a bespoke replacement for inefficient imported logic.

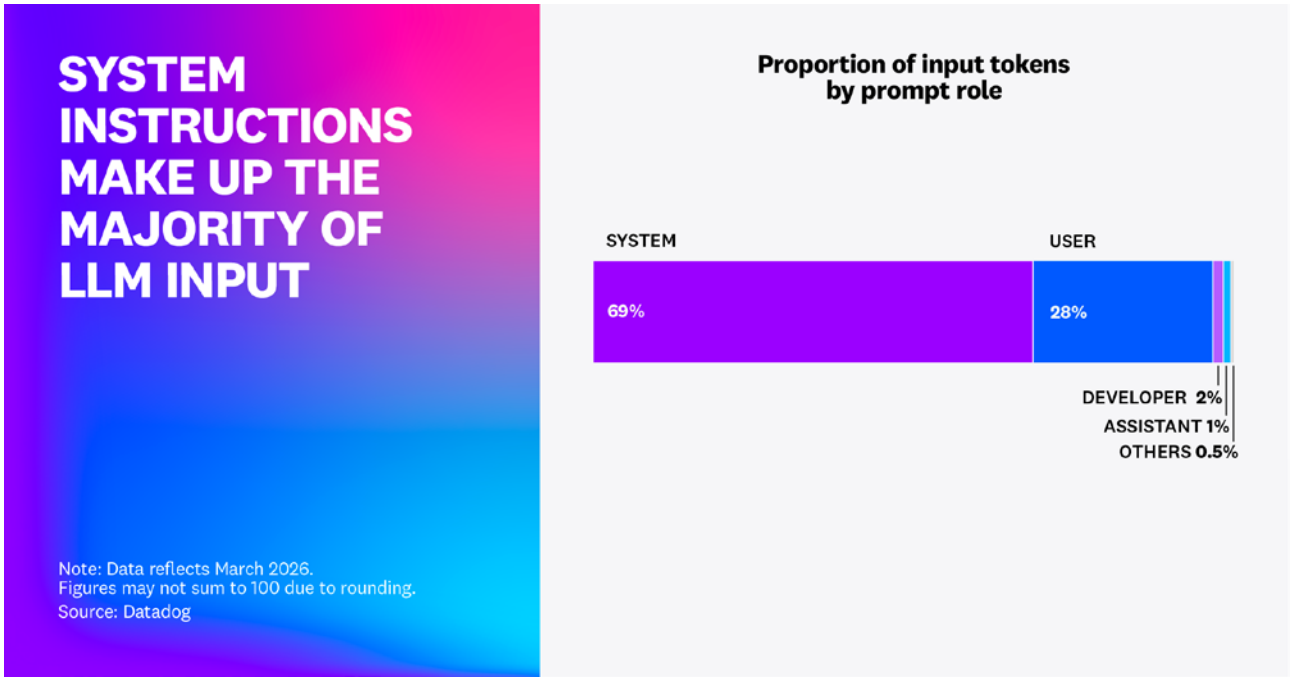
“The next wave of agent failures won’t be about what agents can’t do. It’ll be about what teams can’t observe. Agents need the same production feedback loops we’ve always expected from great software. Unlike traditional software, agents have control flow driven by the LLM itself, which makes observability not just useful, but essential.”

Guillermo Rauch
Founder and CEO at Vercel

FACT 4

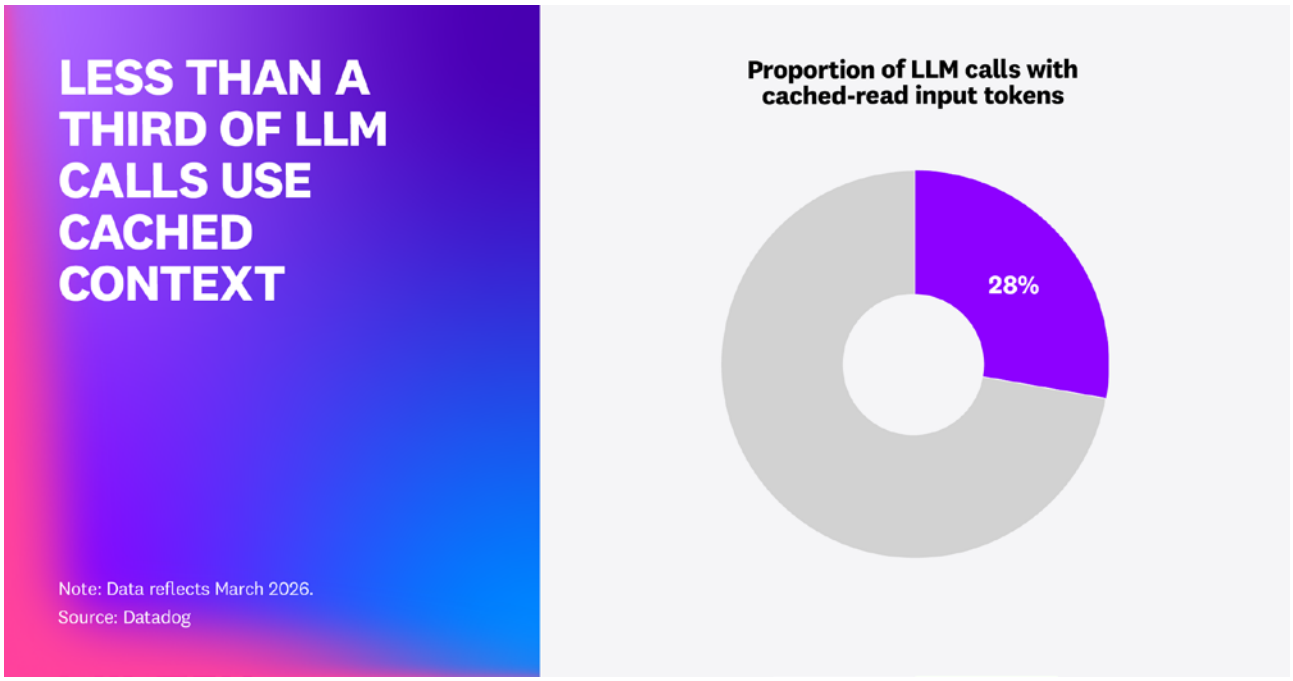
Organizations are building heavily scaffolded agents with large system prompts, but prompt caching is underutilized

Our research found that 69% of all input tokens in customer traces were for system prompts: internal instructions, policy definitions, and tool guidance executing down the chain from the initial user query. This suggests that most context engineering spend among Datadog customers is going toward optimizing repeating system prompts in heavily scaffolded agent systems. Teams should shorten system prompts where possible to reduce token usage and modularize reusable components for caching.



Scaffolded systems require heavier tool usage and more constraints in the form of policy and safety guardrails. When guardrails and tool guidance are often repeated verbatim across calls, a significant cost and latency bottleneck is introduced.

Prompt caching is a highly effective way for teams to reduce cost and increase speed without changing model behavior—especially if the application's stable scaffolding (system instructions, policies, tool schemas) is actually reusable across calls. However, we found that even among models that support prompt caching, only 28% of LLM call spans show any cached-read input tokens. This suggests that the majority of LLM calls in these applications still re-process the full prompt.



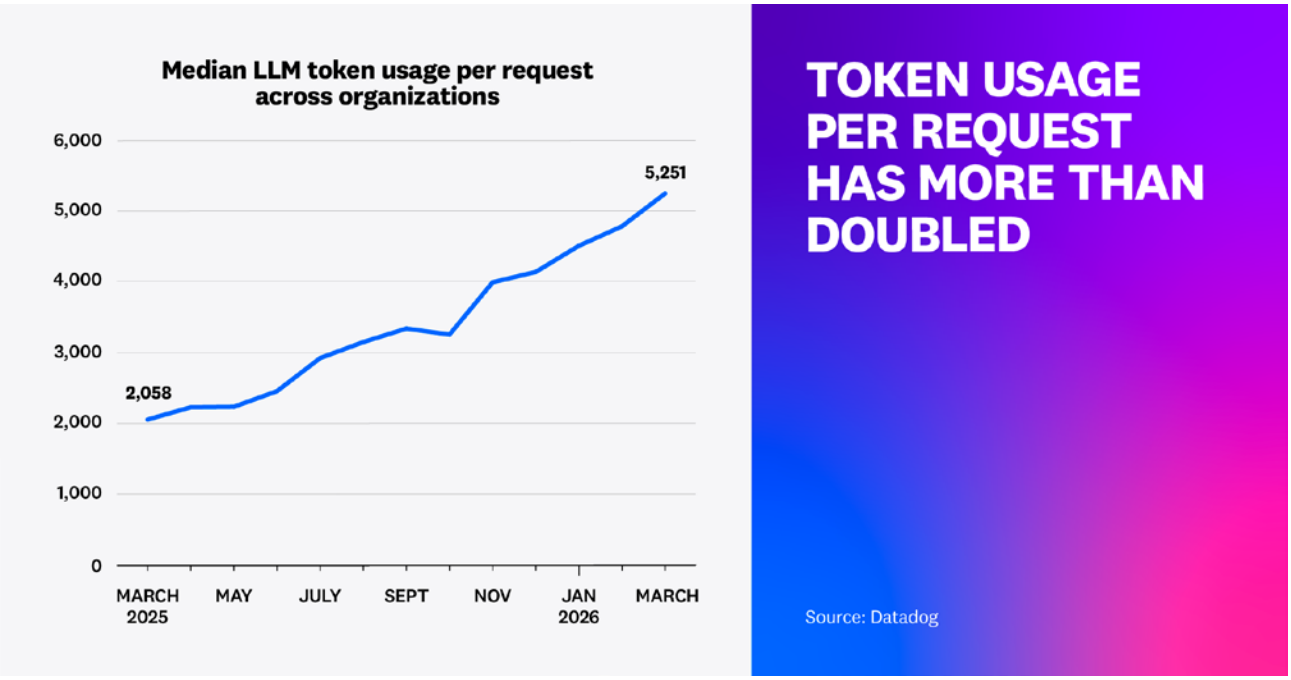
If your app's cache-hit rate or cached-token share is low, the common culprit is prompt layout. If your prompt layout is inefficient, dynamic content could be injected too early or blocks of state that are supposed to be stable could be getting reordered or rewritten between requests—breaking the prefix reuse that facilitates caching.

FACT 5

Context window size has exploded, creating a large potential for context engineering

As agentic AI becomes the industry standard, models become more powerful and large requests become less expensive. Leading models' context windows have moved multiple orders of magnitude over the past two years: from 128,000 tokens to as high as two million tokens in some pricing tiers. This suggests that context windows are no longer a bottleneck for the vast majority of users. Teams are learning to stuff more state—such as conversation histories, retrieved documents, tool outputs, or policy guardrails—into prompts. This context is essential for making agents more reliable and better fit to complex use cases.

Our research examined trace spans for LLM calls across Datadog customers, showing that the average number of tokens used in customers' requests more than doubled for median customers and quadrupled for the 90th-percentile power users year over year.



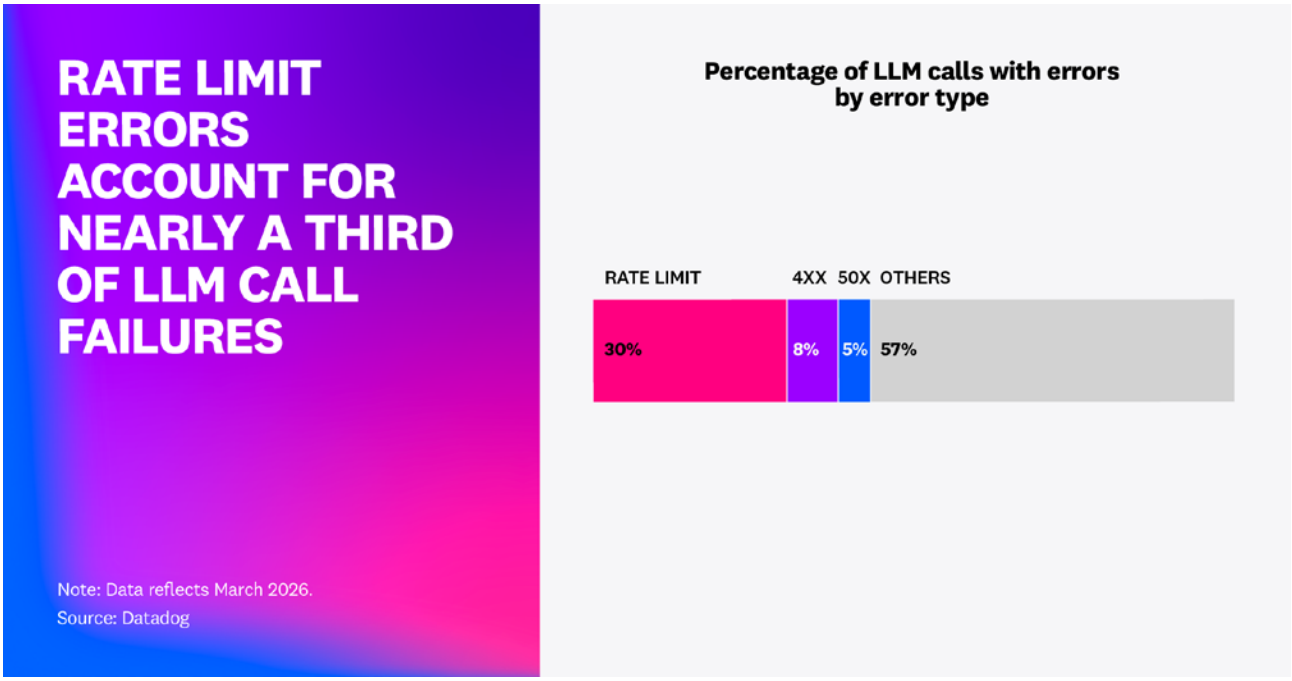
As prompts increase in size and teams find new ways to gather, generate, and inject context into agent pipelines, latency and cost challenges will naturally arise. Further, as prompts grow to include more history, retrieved documents, tool outputs, and guardrails, noise and redundancy can drown out the signal—especially when critical details get buried deep in long inputs.

Context quality—not volume—is the new limiting factor for LLM agents; the majority of teams don't come close to using the full context size of their models. This shifts the core challenge from managing tokens to understanding which information actually drives model decisions. Organizations that invest in context engineering (retrieval quality, summarization, deduplication, and clear information hierarchy) will close the gap between what long-context models allow and what production agents can reliably work with. This means maintaining systems to reliably select, compress, and structure the most decision-relevant information so that the model can make the best possible use of it.

FACT 6

Agent reliability is hitting a capacity ceiling: rate limit errors are the most common LLM call failure

Our research examined LLM call failures in Datadog Agent Observability customer traces. In February 2026, our analysis showed that 5% of all LLM call spans reported an error and 60% of those errors were caused by exceeded rate limits. In March 2026, 2% of all LLM spans in our dataset returned an error and rate limit errors accounted for almost a third of them—nearly 8.4 million rate limit errors in total. This suggests that the capacity ceilings of model providers are leading to compromises in agent reliability. To ensure reliability in dynamic conditions when rate limits are the capacity ceiling for agents, both operational patterns (such as budgeting and backpressure systems) and prompt optimizations are required.



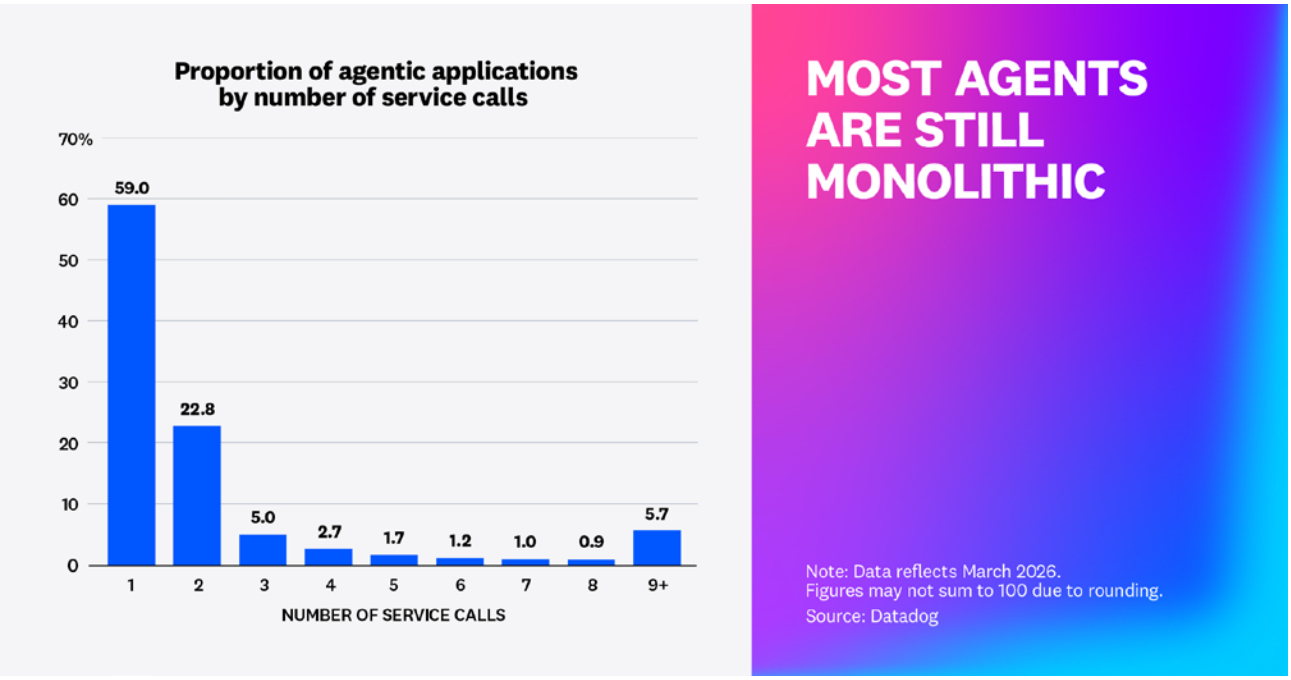
When the dominant production failure mode of LLM applications is capacity, teams need to redouble their capacity engineering efforts. Particularly, capacity quotas shared across an organization and a prevalence of concurrency and retry spikes can lead to cases where periodic bursts of request volume can unpredictably exhaust allocated capacity. This is especially true for systems that run variable loops using ReAct methodologies, for instance, or multiple collaborative agents. The issue compounds when long-lived agent loops hit provider rate limits or organization-specific concurrency caps, which can trigger retries that increase the load further and evolve the issue into a sustained system failure.

Prompts and application logic need to be designed to avoid spikes in loop length and tool fan-out. At the same time, platform teams need to implement queue systems, backoff measures, and fallback capacity into LLM applications' core runtimes. Additionally, by implementing budgets to force agent loops to terminate when a maximum number of calls or tokens are expended, teams can prevent runaway loops that could cause capacity exhaustion to impact downstream services.

FACT 7

Agents are still largely monolithic

Our research found that 59% of agentic application requests only made a single service call, while only 18% of end-to-end agentic application requests made three or more service calls. This suggests that a significant majority of agents are still monoliths. However, other organizations seem to be testing multi-agent architectures or deploying agents on their own services for microservice-style interfacing with the rest of their environments.



Teams know that monoliths don't scale well and are looking to change, but production agents are still largely monolithic. The shift toward dedicated agent services and multi-agent architectures drives new requirements for organizations' platforms. To debug and test these applications, teams need to propagate context and traces across service boundaries. And to manage these distributed platforms, organizations need service maps that include tools.

Looking ahead

Modern AI technology organizations are moving toward multi-model, scaffolded, context-rich, and increasingly distributed systems, where success relies on continuous evaluation of agent behavior, performance, and cost. AI agent architectures are expanding in complexity: Context windows are expanding, prompts are multiplying, and the surface area for invisible drift keeps widening.

This demands a new set of skills teams need to develop: operating reliable evaluation loops, engineering context deliberately, structuring high-signal inputs, and actively governing model and context sprawl before it compounds into technical debt. And through all of it, the fundamentals of operational excellence don't go away—teams still need to enforce budgets and backpressure, build in fallbacks, and route the right tasks to the right models. The next wave of advantage belongs to organizations that can mature their agents into disciplined production systems—continuously evaluating and improving them to be more observable, governable, resilient, and cost-aware.

Methodology

Population

For this report, we compiled usage data from thousands of companies in Datadog's customer base. While Datadog customers cover the spectrum of company size and industry, they do share some common traits. First, they tend to be serious about software infrastructure and application performance. And they skew toward adoption of cloud platforms and services more than the general population. All the results in this article are biased by the fact that the data comes from our customer base, a large but imperfect sample of the entire global market.

Fact 1

For this fact, we looked at the split of organizations working with LLM model providers such as OpenAI, Anthropic, Google, and the rest combined as "Other". We looked at the total sample of organizations that are sending Datadog LLM spans.

Fact 2

For this fact, we looked at the split of organizations using models from different LLM providers. We calculated the percentage of organizations using each model by dividing the total number of organizations sending Datadog LLM spans. We handpicked certain OpenAI, Anthropic, and Google models to highlight trends in the data.

Fact 3

We define frameworks as libraries that manage agent state, tool execution, and control flow. We identified AI framework usage by looking at the dependencies in services with APM instrumentation.

AI frameworks considered include OpenAI Agents, LangGraph, LangChain, Langflow, CrewAI, Microsoft AutoGen, LlamaIndex, CAMEL-AI, MetaGPT, smolagents, Flowise, SuperAGI, Griptape, n8n, Haystack, Spring AI, AgentScope, AgentFlow, Atomic Agents, OpenHands, Prompt Flow, Strands Agents, Letta, Rasa, Lindy, Vercel AI SDK, Botpress, Marvin, Instructor, Guidance, AWS Bedrock Agents, Pydantic AI, Microsoft Semantic Kernel, Mastra, and Chainlit.

Fact 4

For this analysis, we considered all spans from March 2026 and extracted the sender role for each LLM call message. We used character count as a proxy for token count to avoid ingesting sensitive data, approximating token counts by dividing character counts by four. We then summed estimated token counts across all LLM calls and calculated the percentage contribution of each prompt role.

The percentage of LLM calls with cached-read input tokens is defined as the proportion of calls with more than 0 cached-read input tokens, computed only across models that have at least one span containing cached-read input tokens. This restriction ensures the metric is evaluated only for models that support cache reads.

Fact 5

For this fact, we looked at the average tokens per span for organizations sending Datadog LLM traces. Then, we calculated the median tokens per request by organization and visualized this trend over a year.

Fact 6

For this fact, we looked at the status of the LLM calls/spans sent to Datadog in the month of March 2026. If the error status code was 429, then we labeled those spans as rate limit or resource exhausted errors, bucketing other 4xx errors separately. Similarly, transient server and gateway issues are coded as “50x,” and all other error types are bucketed as “Other.”

Fact 7

For this fact, we looked at end-to-end AI agentic applications and the number of LLM calls that an application makes to a service. We only looked at applications with at least one service call.

Data Collection

The above facts are derived from data collected using Datadog’s Agent Observability product and involve analyzing metrics and metadata concerning our customers’ use of LLMs.

Licensing

datadog.com